Integrasi Master Data ERP → Webstore

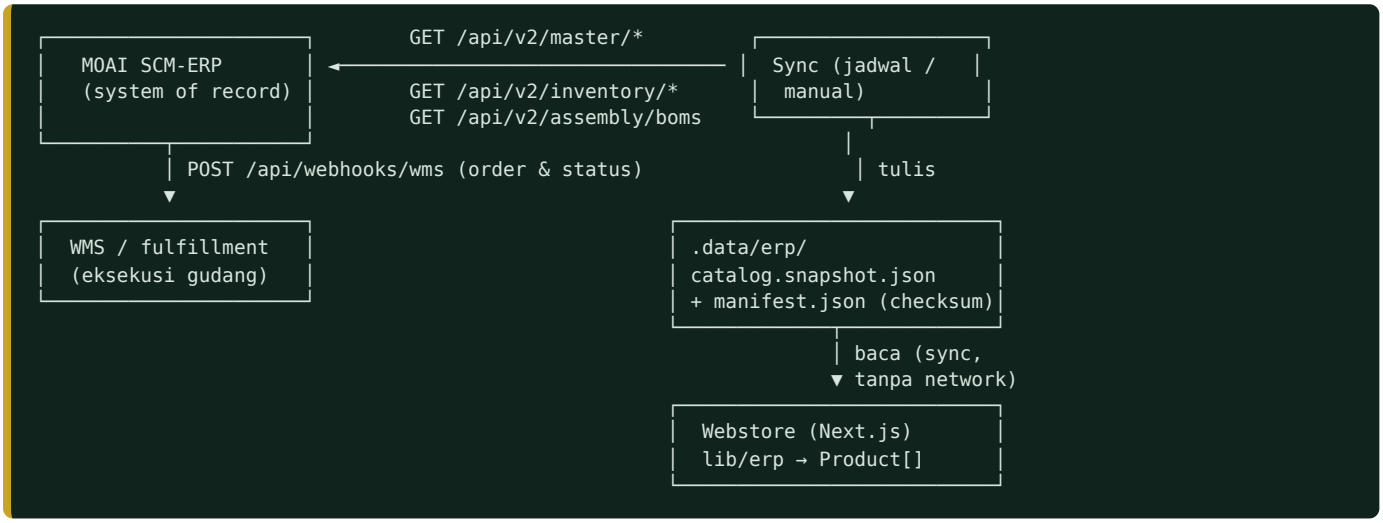
Proyek: OSR / Octarine Sandbox — BPI DMAIC **Versi:** 1.0 · 27 Oktober 2026 **Stack:** Next.js 14 (webstore) · Express + Supabase (MOAI SCM-ERP `scm-erp-pied.vercel.app`)

---

1. Ringkasan

Sebelum perubahan ini, katalog storefront **dibangkitkan di dalam kode**: array wangi, ukuran, dan gudang hardcoded, lalu di-*generate* deterministik menjadi 3.696 SKU sintetis dengan stok kuota palsu. Artinya angka yang dilihat di toko tidak berasal dari sistem mana pun — tidak bisa diaudit dan tidak bisa dipakai sebagai bukti perbaikan proses.

Sekarang katalog, harga, stok, dan komposisi rakitan **berasal dari master data ERP** lewat REST API. Tidak ada satu pun SKU, brand, harga, atau stok yang dipaku di kode.



Pembagian peran yang dipakai:

- **ERP** = *system of record* master data: SKU, nama, brand, kategori, harga, BOM, saldo stok, gudang.
- **WMS** = eksekusi fisik: menerima order, memotong stok, mengirim status.
- **Webstore** = konsumen data. Tidak pernah menyimpan master data sendiri, tidak pernah menulis stok.

---

2. Endpoint ERP yang dipakai

Semua request memakai `Authorization: Bearer <token>` dan header `User-Agent` browser. Base URL: `https://scm-erp-pied.vercel.app`.

Endpoint	Isi	Dipakai untuk
`GET /api/v2/master/products`	278 SKU: `sku`, `name`, `brand`, `category`, `item_type`, `specs`, `stock`, `unit_price`, `wholesale_price`, `is_assembly`, `warranty`	Katalog, harga, atribut
`GET /api/v2/inventory/balances`	`products[]` berisi `sku`, `stock_on_hand`, `allocated`, `available`, `bin_code`	Stok tampil & kuota baseline
`GET /api/v2/assembly/boms`	`target_sku`, `components[]` (`component_sku`, `qty`, `scrap_pct`), `labor_cost`, `overhead_cost`	Komposisi SKU rakitan → stok to-be

Endpoint	Isi	Dipakai untuk
`GET /api/v2/master/warehouses`	`id`, `code`, `name`, `type`	Daftar gudang
`GET /api/v2/master/operational`	`min_stock`, `max_stock`, `reorder_point`	Ambang reorder (opsional)

Catatan operasional:

- **Parameter `limit` / `offset` diabaikan** oleh ERP saat ini — respons selalu penuh. Jangan menulis kode yang mengandalkan pagination sampai ERP mendukungnya.
- Respons `products` berupa **array telanjang**; `balances` berupa **objek** `{ total_sku, total_valuation, products[] }`.
- Wajib kirim `User-Agent` browser-like. UA default tool HTTP (Python urllib / Node fetch) bisa ditolak WAF.
- Token demo (`moai_demo_test`) hanya untuk sandbox. Untuk integrasi nyata buat **service token read-only** (lihat §6).

---

3. Pemetaan field ERP → `Product` storefront

Dilakukan di `lib/erp/mapper.ts`. Tidak ada nilai fallback yang mengarang data: kalau field kosong, yang tampil kosong.

Field storefront	Sumber ERP	Catatan
`sku`	`products.sku`	Kunci utama
`name`	`products.name`	—
`gender` (segmen)	`products.brand`	Di-uppercase; dipakai filter "Brand"
`collection`	`products.item_type`	`PREBUILT` → `prebuilt`, dst.
`sizeLabel`	`products.category`	Ditampilkan di kartu produk
`priceIdr`	`products.unit_price`	Fallback `wholesale_price`
`compareAtIdr`	`wholesale_price`	Hanya diisi kalau > `unit_price`
`baselineQuota`	`balances.stock_on_hand`	Kuota manual = angka ERP apa adanya
`materialCode`	`products.sku`	Dipakai webhook WMS
`notes`	`brand` / `category` / `warranty`	Ditampilkan sebagai atribut master data
`description`	`products.specs`	—
`isBundle`	ada di `boms.target_sku`	—
`components[]`	`boms.components[]`	`multiplierQty` = `qty`
`priceIdr` (bundle)	Σ(`component.unit_price` × `qty`) + `labor_cost` + `overhead_cost`	Harga rakitan dihitung, bukan disimpan

Belum tersedia di ERP: stok per gudang (breakdown). `stockPerWarehouse` sengaja dibiarkan kosong; ERP hanya memberi saldo total per SKU. Kalau nanti dibutuhkan, tambahkan endpoint `GET /api/v2/inventory/balances?group_by=warehouse`.

---

4. Dua pola koneksi

A. Snapshot (default, direkomendasikan)

```
npm run erp:sync   →   .data/erp/catalog.snapshot.json
```

- Storefront membaca file, **tidak pernah** memanggil ERP di request path.
- Build & demo deterministik: angka sama setiap kali dibuka, bisa diaudit lewat `checksum` sha256 di `/admin/simulasi`.
- ERP mati / lambat → toko tetap jalan dengan data terakhir.
- Snapshot **di-commit** ke repo (`.data/erp/` di-whitelist di `.gitignore`), jadi reviewer bisa melihat persis data yang dipakai.

Aktifkan dengan `OSR_CATALOG_SOURCE=snapshot` (default).

B. Live pull

```
OSR_CATALOG_SOURCE=live
```

- Proses menarik ERP saat pertama kali modul katalog dimuat (lazy, di-cache di memori).
- Cocok untuk demo "data benar-benar live" atau saat master data sering berubah.
- Risiko: latensi start + ERP jadi dependency keras. Kalau gagal dan snapshot ada, sistem fallback ke snapshot (kecuali `OSR_ERP_STRICT=1`).

Jangan pakai live pull untuk produksi toko tanpa cache di depan ERP — traffic storefront akan langsung membebani ERP.

---

5. Langkah menghubungkan (0 → 1)

```
cd ~/BPI-Simulation-Webstore

# 1. set kredensial
cp .env.example .env.local
#   OSR_ERP_BASE_URL=https://scm-erp-pied.vercel.app
#   OSR_ERP_TOKEN=<service token ERP>
#   OSR_CATALOG_SOURCE=snapshot

# 2. tarik master data
npm run erp:sync
#   ✓ 278 produk · 278 saldo · 1 BOM · 2 gudang
#   checksum 9af165c10ab26c3b…

# 3. jalankan
npm run dev          # http://localhost:3000
```

Verifikasi cepat:

```
curl -s localhost:3000/api/catalog/products?perPage=2 | head
curl -s localhost:3000/api/catalog/products/SKU-PR-AIO-001
curl -s localhost:3000/api/catalog/stock            # butuh ?sku=
```

Yang harus kelihatan kalau berhasil:

- `/` menampilkan nama produk dari ERP (bukan "Sanctum & Noctis").
- `/shop` filter **Brand** berisi brand nyata (Acer, Asus, …) dengan jumlah SKU.

- `/collections` berisi 3 koleksi hasil `item_type` (`prebuilt`, `barebone`, `spare-part`) dengan hitungan SKU.
- `/admin/simulasi` blok **Jejak integrasi master data ERP**: base URL, waktu snapshot, checksum, jumlah record, dan status integritas referensial.

Refresh berkala

Snapshot adalah cache, bukan sumber. Jadwalkan sync:

```
# tiap 30 menit
*/30 * * * * cd /srv/webstore && OSR_ERP_TOKEN=xxx npm run erp:sync >> /var/log/erp-sync.log 2>&1
```

Setelah sync, restart proses (atau panggil endpoint revalidate) supaya cache memori di-reload. Snapshot dibaca ulang otomatis kalau `mtime` berubah — lihat `lib/erp/snapshot.ts`.

---

6. Yang perlu ditambahkan di sisi ERP

Saat ini webstore memakai endpoint internal `/api/v2/master/*`. Untuk integrasi yang benar, tambahkan **permukaan integrasi terpisah** supaya token storefront tidak punya akses ke endpoint tulis:

```
GET /api/v2/integration/catalog        # produk + harga + atribut (read-only)
GET /api/v2/integration/stock          # saldo per SKU (+ ?updated_since=)
GET /api/v2/integration/boms           # BOM aktif saja
GET /api/v2/integration/health         # untuk monitoring
```

Aturan minimal:

1. **Auth terpisah.** Token service dengan scope `catalog:read` saja. Jangan pakai token demo, jangan pakai service-role Supabase.
2. **Rate limit** per token (mis. 60 req/menit) + header `Retry-After`.
3. **`updated_since` + `ETag`.** Storefront cukup menarik delta, bukan 278 SKU penuh setiap kali.
4. **CORS** dibatasi ke origin webstore saja (kalau dipanggil dari browser; dengan pola snapshot tidak perlu CORS sama sekali).
5. **Audit log** setiap akses integrasi: token, endpoint, jumlah record.

Contoh route Express (pola yang dipakai ERP ini):

```
// server/routes/integration.js
router.get("/integration/catalog", requireScope("catalog:read"), async (req, res) => {
  const since = req.query.updated_since ?? null;
  const q = supabase
    .from("v2_store")
    .select("data")
    .eq("collection", "products")
    .order("updated_at", { ascending: true });
  if (since) q.gte("updated_at", since);

  const { data, error } = await q;
  if (error) return res.status(500).json({ error: error.message });

  const items = data.map((r) => r.data);
  res.set("Cache-Control", "private, max-age=60");
  res.json({ count: items.length, updated_at: new Date().toISOString(), items });
});
```

`requireScope` mengecek tabel `api_tokens` (hash token, scope, `revoked_at`).

---

7. Arah sebaliknya: order & status (webstore → WMS)

Integrasi tidak berhenti di katalog. Alur pesanan sudah ada di `/api/webhooks/wms` dan didokumentasikan di `INTEGRASI_WEBHOOK_ERP.md`:

1. Checkout di webstore → `POST /api/webhooks/wms` (`event: ORDER_CREATED`, `data.material_code` = **SKU ERP**, bukan lagi `M###`).
2. WMS memproses, memotong stok, lalu mengirim `STOCK_ADJUSTED` / `ORDER_STATUS`.
3. Webstore mengurangi stok tampil dari event tersebut (`lib/state.ts`, `soldSnapshot`).

Perubahan yang sudah dilakukan: validasi `material_code` sekarang menerima SKU ERP (`SKU-PR-AIO-001`, `PB-ARES-78X`) dan masih menerima format lama `M###`.

---

8. Temuan data saat integrasi pertama

Hasil `npm run erp:sync` pada master data ERP saat ini:

a. BOM yatim (orphan). Satu-satunya BOM menunjuk `PB-ARES-78X`, tetapi baik target maupun 7 komponennya (`SP-CPU-7800X3D`, `SP-MB-B650PLUS`, `SP-RAM-DDR5-32G`, `SP-SSD-990P-1T`, `SP-GPU-RTX4070`, `SP-PSU-RM750E`, `SP-CASE-CC560`) **tidak ada** di `master products`. BOM ini tidak bisa dihitung stoknya — persis mode kegagalan "komposisi tidak tersedia" pada FMEA F3. Webstore menampilkannya sebagai `Data komposisi tidak tersedia` dan menandainya di panel integrasi.

b. Stok seragam 133. Seluruh 278 SKU memiliki `stock_on_hand = 133`. Ini pola seed, bukan hasil operasi. Untuk simulasi DMAIC ini masih sah (baseline manual), tapi angka absolut tidak boleh diklaim sebagai data operasional nyata.

c. Tidak ada breakdown per gudang. 2 gudang terdaftar, tapi saldo hanya total per SKU.

Implikasi ke paper: klaim "3.696 SKU / 7.325 baris stok" berasal dari dataset Octarine/Perpack, **bukan** dari ERP ini. Kalau paper memakai ERP sebagai sumber, angka harus di-regenerate (278 SKU + 1 BOM yatim) atau master data ERP diisi dulu dengan dataset yang dimaksud. Jangan mencampur dua sumber angka dalam satu bab.

---

9. Troubleshooting

Gejala	Penyebab	Perbaikan
`ERP menolak request (401)` di `erp:sync`	Token kosong / salah	Set `OSR_ERP_TOKEN` ke service token ERP
`ERP 404`	Base URL salah atau endpoint berubah	Cek `OSR_ERP_BASE_URL`, uji `curl -i <base>/api/health`
Snapshot tidak ada saat `next start`	Belum pernah sync / `.data` terhapus	`npm run erp:sync`, lalu restart
Data lama tetap tampil setelah sync	Cache memori per proses	Restart proses; snapshot hanya reload saat `mtime` berubah
Semua stok 0	`balances` gagal dimuat, fallback ke field `stock`	Cek endpoint `inventory/balances`
`Data komposisi tidak tersedia` di SKU rakitan	BOM yatim / komponen tidak ada	Perbaiki BOM di ERP (lihat §8a)

Gejala	Penyebab	Perbaikan
Filter brand kosong	`brand` null di semua produk	Isi brand di master produk

---

10. Berkas yang terlibat

```
lib/erp/
  types.ts      DTO mentah ERP + tipe snapshot
  pull.mjs      puller HTTP (retry, timeout, integrity, checksum)
  pull.d.mts    deklarasi tipe untuk puller
  client.ts     pintu masuk: snapshot vs live, error yang bisa dibaca
  snapshot.ts   baca/tulis cache + reload saat mtime berubah
  mapper.ts     ERP → Product[] + koleksi + segmen + metadata
scripts/
  erp-sync.mjs  CLI `npm run erp:sync` (diff checksum & temuan integritas)
lib/catalog.ts  akses katalog storefront (tanpa data hardcoded)
.data/erp/
  catalog.snapshot.json   payload + integrity + checksum (di-commit)
  manifest.json           ringkasan tanpa payload (untuk diff)
.env.example    OSR_ERP_* dokumentasi variabel
```